

- 学习目标
- 这篇讲义怎样读
- 从单次请求到 Agent 任务，系统责任发生了什么变化
 - Function Calling 和 Agent Loop 是转折点，但还不是终点
- 一个真实的长任务案例：同一个模型，为什么循环跑仍然不够
 - Anthropic 改变了什么
- Harness 这个词到底在说什么
 - 常见视角与代表来源
 - 先统一三个概念
- 四种常见说法分别位于哪一层
- 一个可操作的边界判断方法
 - 四问的完整含义
 - 练习：用四问法判断常见对象
- Agent 任务会带来哪些新问题
 - 任务开始前：目标与环境没有被说清楚
 - 行动过程中：非确定性决策遇到有副作用的现实系统
 - 长任务与服务化阶段：聊天记录无法承担全部状态
- 从问题反推主要模块
 - 主要模块：是什么，解决什么问题
- Harness 覆盖哪些层次
 - 分层解释
- 它不是 Prompt Engineering，也不只是 Agent Loop
- 每个场景都要考虑 Harness，但不需要同样厚
- 三个工程观察：为什么不能只看模型
 - 观察 1：固定模型下改变接口，任务结果会跟着变
 - 观察 2：强模型 + Loop + Compaction 仍无法自然维持长任务
 - 观察 3：OpenAI 的 Agent-first 仓库把“模型看不见的东西”改造成系统对象
 - 三个观察放在一起，能得到什么
- 从三个观察得到的四条工程结论
- 所以 Harness 工程应该掌握什么
- 总结

Harness Engineering

从模型调用到 Agent 任务系统

学习目标

本节围绕一个问题展开：**当模型不再只是给出一段回复，而是要在真实系统里持续完成任务时，模型周围需要什么？**

学完本节后，应当能够：

- 解释 Harness Engineering 的主流定义为什么不一致；
- 判断“执行空间、文档规范、组织变革、自建框架”分别位于哪一层；
- 说明 Agent 任务为什么会带来单次模型调用中没有的问题；
- 从任务问题反推出运行与控制系统的主要模块；
- 区分 Harness 与 Prompt Engineering、Agent Loop、MCP、Eval Harness、组织变革；
- 看懂长期记忆、自进化、多 Agent 与 ACP 在整张系统地图中的位置。

这篇讲义怎样读

阅读时可以抓住三条线：

1. **定义为什么会分歧**：同一个 Harness，有人说的是运行时，有人说的是仓库外层，也有人说的是组织交付方式。
2. **边界怎样判断**：一个对象是不是完整 Harness，不能只看名字，要看它是否具备行动、状态、控制和验证闭环。
3. **问题怎样反推模块**：Agent 任务遇到的每个问题，都能反推出一类系统责任，例如状态、权限、服务、观测或恢复。

只要能把一个具体 Agent 任务放到这三条线上，就不会被术语绕晕。

从单次请求到 Agent 任务，系统责任发生了什么变化

单次请求的主要风险是“回答得不好”。Function Calling 之后，模型的输出开始代表一个行动请求；进入 Agent Loop 后，行动会根据观察结果持续发生；进入长任务后，工作空间、任务状态、副作用、验证和恢复都需要跨越多轮甚至多个会话。

可以用“模型、数据、控制”三部分来理解这种变化：

- **模型**提供理解、判断、规划和表达能力；
- **数据**补充任务所需的知识和现实状态；
- **控制**把模型的判断组织成可执行、可持续、可验证的任务过程。

Harness Engineering 主要讨论第三部分，并把“控制”从输出格式和一次工具调用，扩展到整个 Agent 任务生命周期。



图 1：从单次请求走向 Agent 任务，系统责任逐层扩大。

Function Calling 和 Agent Loop 是转折点，但还不是终点

阶段	模型输出代表什么	系统新增责任
单次请求	一段文本或结构化结果	输入、输出格式、超时、基本审计
Function Calling	一个外部行动请求	工具选择、参数校验、执行、错误回传
Agent Loop / ReAct	下一步行动决策	循环、观察、重试、停止条件、预算
Agent 任务	对真实目标的持续推进	工作空间、状态、记忆、权限、验证、恢复、服务化

模型的能力没有消失。变化在于：任务结果不再等于模型最后生成的文字，结果越来越多地存在于文件、数据库、浏览器、工单、代码仓库和其他外部系统里。

一个真实的长任务案例：同一个模型，为什么循环跑仍然不够

案例背景：Anthropic 的跨上下文长任务实验

来源说明 Anthropic Engineering, 2025-11-26, *Effective harnesses for long-running agents*

- **实验对象：**Claude Agent SDK 中的前沿编码模型 Opus 4.5；系统已经具备工具调用和上下文压缩（compaction）。
- **任务方式：**给模型一个高层目标，例如构建一个 `claude.ai` 风格的完整 Web 应用；当一个上下文窗口结束后，用新会话继续循环推进。
- **要回答的问题：**有了强模型、工具、Agent Loop 和上下文压缩，为什么长任务仍然不能自然变成稳定交付？

稳定出现的失败模式：

1. Agent 试图一次完成太多工作，上下文耗尽时留下半实现、未说明的代码；
2. 新会话不知道上一会话到底做了什么，只能从仓库现状重新猜；
3. 项目已有部分功能后，后续会话容易把“已有进展”误判成“全部完成”；
4. 环境已经损坏时，Agent 仍继续增加功能，使错误继续累积。

这个案例说明：循环和上下文压缩只解决“继续调用模型”，并没有自动解决目标分解、跨会话状态、工作基线、完成证据和交接。

阅读入口：[Anthropic — Effective harnesses for long-running agents](#)

这里不用把重点放在某个数字上。它真正有用的地方，是把长任务失败的机制暴露得很清楚：模型能继续工作，不代表任务系统能持续交付。

真实案例：长任务不是把 Agent Loop 多跑几次

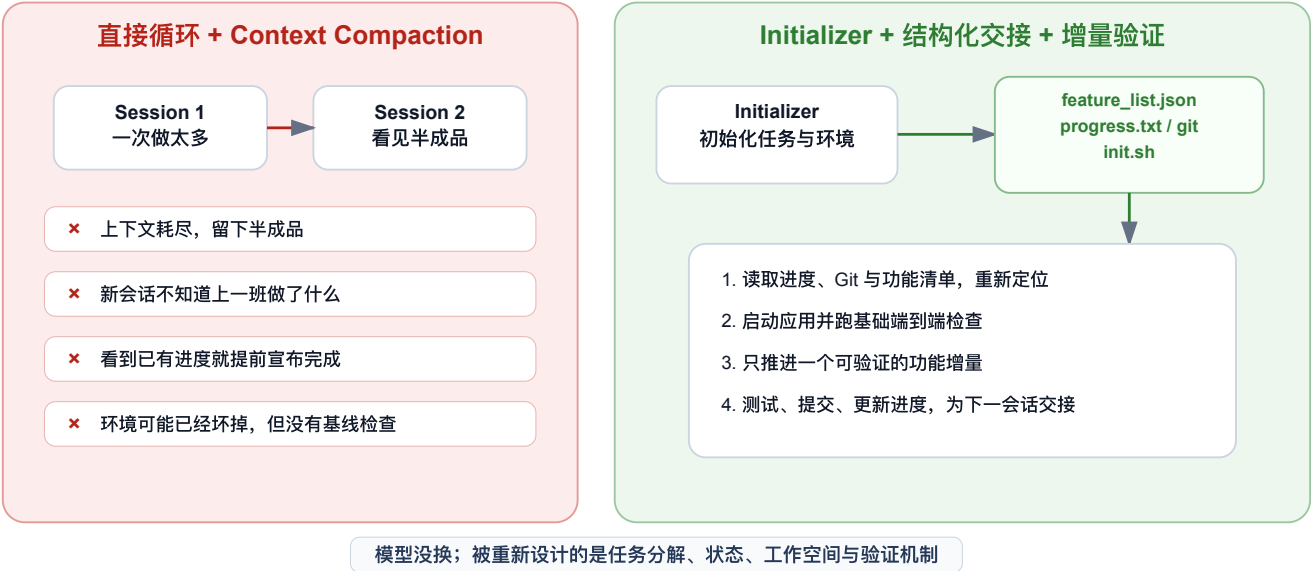


图 2：问题不是“循环次数不够”，而是跨会话的任务结构、状态和验证没有被外部化。

Anthropic 改变了什么

他们没有重新训练模型，而是重做模型周围的任务结构和运行约束：

观察到的失败	加入的系统工件或规则	为什么有用
一次做太多、上下文中断	初始会话把高层目标展开为完整功能清单；后续会话一次只推进一个功能	把“整个项目”改造成有限、可选择、可检查的下一步
新会话失忆	claude-progress.txt 与 Git 历史	新会话读取外部事实，而不是依赖压缩后的聊天摘要猜测
不知道怎样启动和检查系统	initializer 生成 init.sh；新会话先启动应用并做基础端到端检查	先确认当前基线可工作，再增加新变化
提前宣布完成	功能清单初始全部标记为 failing；实际端到端测试后才允许改为 passing	“完成”由外部验收结果决定，不由模型一句话决定
环境留下半成品	每次会话结束前提交代码、记录进度并保持可继续工作的基线	让恢复、回滚和下一次交接都有明确落点

这个案例把 Harness 问题说得很具体：模型负责提出和执行下一步，但任务能否持续推进，取决于模型之外怎样管理目标、环境、状态、证据和交接。

► 案例里的具体工件

- `feature_list.json`: 把高层需求展开成可验证的端到端功能；示例里可以细到上百项，并在开始时全部标记为 failing。
- `claude-progress.txt`: 记录每个会话做过什么、剩下什么。
- Git 历史: 保存可恢复的工作基线和变更事实。
- `init.sh`: 让后续会话不必重新猜测怎样启动应用。
- Puppeteer MCP / 基础端到端测试: 新会话先确认基础功能仍然可用，再开始新工作。

这些工件不是“记忆插件”的堆叠，而是把任务状态从聊天上下文迁移到可见、可版本化、可验证的外部世界。

Harness 这个词到底在说什么

“Harness”来自传统软件工程里的 test harness：它不是被测对象本身，而是为被测对象提供输入、运行环境、观测和断言的一套外部装置。Agent harness 沿用了这个隐喻，只是对象从“被测程序”变成了“需要持续观察、行动和修正的模型”。

公开讨论里没有一个已经稳定下来的唯一口径。不同作者分别在回答不同问题：

- 一个 Agent 产品里，除了模型还包含什么？
- 什么条件下，一个系统才算完整的 Agent Harness？
- 使用现成 Coding Agent 时，团队怎样在外层提高质量？
- 每次失败怎样变成下一次不会再犯的工程改进？
- 当 Agent 成为主要生产者后，仓库和组织怎样变化？

“Harness Engineering”公开讨论中的五种分析单位



图 3：定义分歧主要来自分析单位不同，而不是谁对谁错。

常见视角与代表来源

下面这些视角不需要互相排斥。它们经常都叫 Harness，只是分析单位不同：有的看模型周围的全部非模型层，有的只看运行时，有的看仓库和团队外层。

视角	代表来源	它把 Harness 放在哪里	适合解释什么	容易误读的地方
广义非模型层	Vivek Trivedy / LangChain, <i>The Anatomy of an Agent Harness</i>	除模型外的代码、配置和执行逻辑，包括 prompt、工具、Skills、MCP、文件系统、Sandbox、编排和中间件	解释“Agent 产品除了模型还需要什么”	用于边界判断时过宽，因为几乎所有非模型部分都会被纳入
严格运行时	Sanderson Oliveira de Macedo, <i>What makes a harness a harness</i>	任务运行期间的循环、环境行动接口、任务感知上下文和独立控制	判断一个对象是否足够完整	容易被误读成唯一标准；更适合作为边界测试工具
运行时基底	Hailin Zhong、Shengxin Zhu, <i>AI Harness Engineering</i>	任务规格、上下文、工具、项目记忆、任务状态、可观测、验证、权限等责任	解释 Harness 覆盖哪些系统责任	容易把责任清单误读成功能采购清单
执行与托管运行时	Microsoft Agent Framework；Anthropic 长任务案例	Shell、文件系统、审批、长会话上下文、托管执行环境	解释模型怎样接到真实执行	容易只看到工具环境，忽略状态和验证
使用者外层 Harness	Birgitta Böckeler / Martin Fowler, <i>Harness engineering for coding agent users</i>	文档、Skills、Lint、测试、Review Agent 等 Guides 与 Sensors	解释现成 Coding Agent 外面还要补什么	默认内层 Agent 产品已经有循环和工具

视角	代表来源	它把 Harness 放在哪里	适合解释什么	容易误读的地方
失败防复发机制	Mitchell Hashimoto, <i>My AI Adoption Journey</i>	每次重复失败都沉淀为规则、工具或回归检查	解释 Harness Engineering 的持续改进	它是改进方法，不是完整 Harness 本体
Agent-first 交付系统	Ryan Lopopolo / OpenAI, <i>Harness engineering: leveraging Codex in an agent-first world</i>	仓库知识、计划、CI、架构约束、日志指标和清理机制都围绕 Agent 可读性重构	解释 Harness Engineering 怎样扩展到交付系统和组织方式	不能把全部组织变革都等同为运行时
最小自建路线	Mario Zechner / Pi, minimal coding agent 相关文章与文档	很小的核心、少量通用工具和扩展机制	说明“自建”是一种实现路线	自建不是必要条件；配置现成系统也可以形成 Harness

► 读这张表时注意

- 同一个组件可以出现在多个视角里。比如 **AGENTS.md** 可以是指导文档，也可以是失败防复发的一部分。
- 同一个词的边界会随问题变化。讨论产品责任时可以用宽口径，讨论是否“完整”时要用严格口径。
- 这里列代表来源，是为了让判断可追溯；课堂讨论仍然围绕概念边界和系统责任，不围绕资料本身。

推荐的三层口径：先区分对象，再讨论边界



本课程系列的技术重点在中间层；英文资料常写 Agent Harness / Runtime Harness / Runtime Substrate

图 4：先区分 Agent 的运行核心、项目外层和组织系统，讨论会清楚很多。

先统一三个概念

行业语境里常见 Agent Harness、Runtime Harness、runtime substrate 等叫法，边界并不完全一致。为了避免同一个词在不同层次之间跳动，可以先把三个对象分开：

1. **Agent Harness / Agent 运行与控制系统**：把模型变成能够在环境中持续行动、同时受到约束和验证的系统；
2. **Harness Engineering**：设计、评估、部署并持续改进这套系统的工程工作；
3. **Agent-first Operating Model**：围绕 Agent 重构仓库、交付流程、角色与责任的组织方式。

三者相互依赖，但不是同义词。这里主要讨论第一个对象，并说明 Harness Engineering 如何围绕它展开。

四种常见说法分别位于哪一层

说法	判断	代表性依据	判定理由
为模型 高质量 工作提 供一套 执行空 间	属于核心方向， 但执行空间本身 不等于完整 Harness	Microsoft 把 Shell、文件系 统、审批和上下文管理称为 Harness 构件；Anthropic 案 例把 Git、启动脚本、浏览 器测试作为长任务环境	Workspace、Sandbox、 Shell、浏览器和网络让模 型能够行动，也能提供隔 离；但没有自适应循环、 任务状态和完成验证时， 它们仍只是环境组件
为模型 高质量 工作建 立一套 执行文 档规范	属于指导/上下 文层，也可以成 为用户外层 Harness 的一部 分	OpenAI 的 agent-first 案例试 过巨型 AGENTS.md ，后来 改成短目录 + 结构化 docs/ + CI；Böckeler 把文档和 Skills 归为 Guides	文档只有被稳定发现、按 需加载、版本化，并与测 试、Lint、状态检查等反 馈机制联动，才从“说明 书”进入 Harness 闭环
为基于 模型的 工作获 得高质 量产出 而进行 组织变 革	通常属于更外层 的 Agent-first Operating Model ，不与 Harness 同义	OpenAI 的 agent-first 案例把 人放在意图、环境和反馈回 路设计的位置；Böckeler 把 团队文档、Skills、测试和 Review Agent 归入使用者外 层 Guides 与 Sensors	角色、流程和治理本身属 于组织设计；当权限、审 批、证据要求、预算和升 级规则被编码进运行系统 时，其中一部分才成为 Harness
在模型 基础上 自建工 程运行 框架	可以属于，但 “自建”不是必要 条件	Pi 把最小核心 + 通用工具 + 扩展称为 Harness； Microsoft、LangChain 等提 供可配置的现成原语和运行 产品	判断标准是是否形成行 动、上下文/状态、控制和 验证闭环，而不是代码是 否从零编写

因此，MCP Server、ACP、一个 Skill 文件、一个 Sandbox、固定 Workflow、**AGENTS.md** 或 Eval Harness 都可能非常重要，但单独出现时通常只是 **Harness** 的组件、邻接系统或外层机制。

一个可操作的边界判断方法

判断一个对象是不是完整 Harness，可以不用先争名字，而是问四个运行时问题。这个“四问法”来自 Sanderson Oliveira de Macedo 2026 年的概念预印本 *What makes a harness a harness*，这里把它当作边界判断工具，而不是行业标准。

完整的 Agent Harness 大概是这样一层东西：它包在一个或多个模型周围，用行动循环、环境工具、上下文管理和独立控制，把模型变成能在外部环境中完成任务的 Agent。

这句话拆开之后，就是下面四问：

- 有没有运行时闭环；
- 能不能通过接口感知并改变环境；
- 有没有任务感知的上下文和状态管理；
- 有没有不依赖模型自觉的控制或验证。

阅读入口：[arXiv 2606.10106](#)

这四问适合做边界测试，不适合当成功能优先级清单。一个系统不完整，不代表它没用；它可能只是某个关键组件。

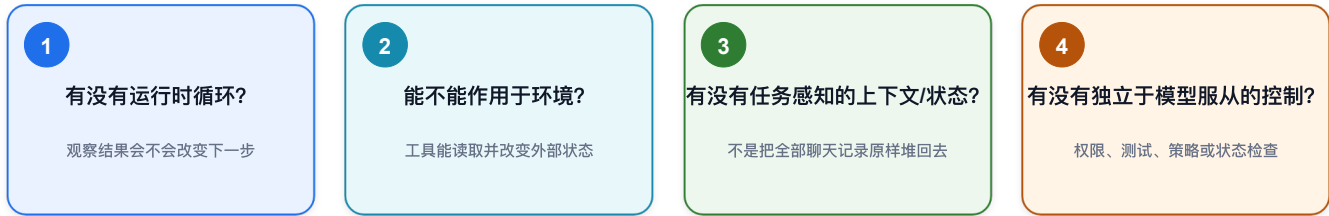
四问的完整含义

问题	通过条件	为什么是必要的	通过示例	不通过示例
T1：运行时是否存在“推理/决策—行动—观察”的闭环？	上一步的环境观察能够改变下一步行动，而不是永远按固定清单执行	没有闭环，系统只是单次生成器或固定流水线，不能根据现实反馈持续推进	运行测试后根据失败信息决定修改哪段代码	固定执行“检索→生成→格式化”，观察结果不能改变路径
T2：模型是否能够通过接口感知并改变外部环境？	工具不只返回知识，还允许写文件、运行命令、操作浏览器或调用业务系统	Agent 任务的结果存在于外部世界；如果模型只能写建议，它仍是助手而不是执行系统	编辑仓库、创建工单、更新数据库（受权限约束）	只给出修改建议文本，不实际改变环境
T3：系统是否主动、按	进入/离开模型窗口的信息由	多步任务会不断产生工具结果和	从 Progress、Plan、当前失败	每轮原样追加全部聊天

问题	通过条件	为什么是必要的	通过示例	不通过示例
任务内容管理上下文和状态？	当前任务和观察决定，而不仅仅是把全部历史堆回去或按长度截断	状态；没有选择机制，相关信息会被噪声淹没	日志中选择下一步所需信息	记录；超长时机械删除最早消息
T4：是否至少有一种不依赖“模型自觉服从”的控制或验证？	权限、预算、测试、策略门、状态断言等机制能独立阻断、改变或确认执行	Prompt 中的“不要做”仍取决于模型听话；系统需要可强制的边界和独立完成证据	工具调用上限、只读权限、单元测试、审批门、幂等键	只记录日志，或只在 Prompt 里要求“请勿删除文件”

两个容易误读的阈值：T3 不是“有聊天历史”就算，而是任务感知的选择；T4 不是“模型说自己检查过”就算，而是控制效果不依赖模型是否合作。严格口径下，T2 也不只是“能看见环境”；接口还应允许改变环境。

四问法：判断一个对象是不是完整的 Agent Harness



四项都“是”时，才适合称为完整 Harness；这是 2026 年预印本提出的研究工具，不是行业标准

图 5：四项都满足，才适合称为完整的 Agent Harness。

练习：用四问法判断常见对象

这里不需要写代码。四问法本质上是一张边界评估表：逐项看候选对象已经满足什么、还缺什么，再给出严格口径下的判断。

对象	已满足	主要缺口	严格判定	该怎么看
单次 Prompt 模板	无	T1 运行时循环、 T2 环境行动、 T3 任务感知上下文、T4 独立控制	组件 / 邻接系统	它影响模型表达，但不负责执行和验证
仅 AGENTS.md / CLAUDE.md	无	T1、T2、T3、T4	组件 / 邻接系统	它是规则入口；需要运行系统读取、选择和执行
MCP Server / 工具集合	T2 环境行动	T1 运行时循环、 T3 任务感知上下文、T4 独立控制	组件 / 邻接系统	它提供行动接口，但不决定任务怎样推进
Sandbox / 容器	T2 环境行动、T4 独立控制	T1 运行时循环、 T3 任务感知上下文	组件 / 邻接系统	它控制环境和权限，但不负责规划、记忆和交接
固定 DAG 工作流	T2 环境行动、T4 独立控制	T1 运行时循环、 T3 任务感知上下文	组件 / 邻接系统	它能执行固定路径，但观察结果通常不能改变整体策略
仅 while loop + tools (无持久状态 / 验证)	T1 运行时循环、 T2 环境行动	T3 任务感知上下文、T4 独立控制	组件 / 邻接系统	它看起来像 Agent，但长任务会缺状态、权限边界和完成验收
成熟 Coding Agent 产品 (完整配置)	T1、T2、T3、T4	无	完整 Harness	产品内部已经把循环、工具、上下文、权限和验证机制组合起来
Agent + 状态 + Policy + Tests	T1、T2、T3、T4	无	完整 Harness	自建系统也可以完整，前提是四类责任都有人承担

关键观察：四问法最有用的地方，不是给对象贴“高级/低级”的标签，而是把缺口说清楚。

- MCP Server、Sandbox、固定 DAG 工作流都可能是关键组件，但单独看通常缺少运行时闭环、任务感知上下文或完整验证；

- 仅有 **while loop + tools** 看起来很像 Agent，但没有持久状态和独立验证时，长任务很容易把错误一路带下去；
- **Agent Framework / SDK** 提供构建原语。只有被实例化成具体运行系统后，才可能成为 Harness；
- **Evaluation Harness** 负责运行任务、记录轨迹和评分。它在“运行 Agent”，但不等于 Agent 自己执行业务任务时使用的 Harness。

四问法解决的是“某个对象是否足够完整”。评估完边界之后，还要把问题放回任务生命周期里看：一个 Agent 任务从接入到结束，会在哪些地方需要运行与控制系统介入。

Agent 任务会带来哪些新问题

Agent 任务与普通模型调用的差异，不只是“步骤更多”。它改变了系统需要负责的对象：

1. **输出变成行动**：一个 token 序列可能对应文件修改、转账、工单或外部消息；
2. **一次调用变成持续循环**：错误会累积，系统需要预算、停止、重试和升级；
3. **外部环境成为任务状态的一部分**：文件、进程、浏览器、数据库都可能在变化；
4. **上下文窗口不再等于任务记忆**：会话会压缩、中断、重启或并发；
5. **最终回复不再等于交付结果**：完成需要独立证据；
6. **调用接口变成长期服务**：需要消息、流式输出、取消、并发、身份和审计。

Agent 任务不是一次调用，而是一段有状态的生命周期

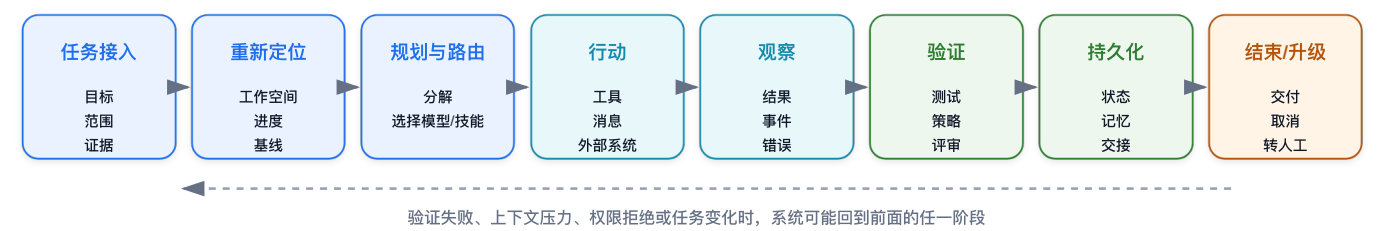


图 6：Harness 的职责分布在任务全生命周期，而不是只包在模型 API 外面。

任务开始前：目标与环境没有被说清楚

常见问题	发生原因	系统保障
只写“做什么”，没有“什么算完成”	验收条件留在人脑里	任务契约：意图、范围、验收证据、预算、升级条件

常见问题	发生原因	系统保障
Workspace 已有半成品，Agent 却按空项目理解	模型只看到局部上下文	Orient：读取目录、版本、进度、基线测试与未完成列表
模型能看到工具，却不知道权限边界	身份和授权不是语言问题	运行时绑定身份、最小权限、网络和数据范围
一开始加载全部文档和工具	上下文噪声和选择负担过高	渐进式披露：按任务加载 Skills、知识和工具定义

行动过程中：非确定性决策遇到有副作用的现实系统

常见问题	发生原因	系统保障
选错工具或参数	工具语义不清、功能重叠	清晰命名、Schema、类型校验、可行行动错误信息
超时后重复执行写操作	Agent 不知道前一次是否部分成功	幂等键、状态查询、事务、重试策略、回滚
循环持续但没有进展	模型在局部反馈里重复同一策略	step / time / cost budget、no-progress 检测、重新规划
试图执行高风险动作	Prompt 里的“不要”不具强制力	Policy gate、审批、只读/可逆/不可逆分级
工具输出淹没上下文	外部结果过大且未经整理	过滤、分页、截断、摘要与 Context manifest

长任务与服务化阶段：聊天记录无法承担全部状态

常见问题	发生原因	系统保障
Context reset 后重复劳动	决策和进度只存在于聊天历史	Progress、Plan、Decision log、Checkpoint、Handoff
进程崩溃后从头开始	环境与任务状态没有一致快照	Workspace snapshot、可重放事件、恢复状态机

常见问题	发生原因	系统保障
外部系统持续发 来消息	单次 request/response 模型 不适合异步会话	会话身份、消息收发、队列、并 发控制与取消
流式输出和后台 任务无法关联	运行过程没有统一事件模型	Event stream、Run ID、状态事 件、消费端协议
模型宣布完成但 结果不完整	生成者同时充当验收者	Tests、状态断言、独立 Reviewer、人工升级
失败无法复现和 改进	只保存最终回复	Trace、工具调用、环境快照、 Harness 版本与 Eval 回归

从问题反推主要模块

下面这张图不是某个产品的模块结构，而是从 Agent 任务生命周期抽象出的责任地图。具体实现可以拆分或合并模块，关键是每项系统责任都有明确归属。



图 7：输出控制、编排、工具、Skills、Workspace、记忆、服务、消息、流式、日志和 Eval 可以放进同一张责任地图。

主要模块：是什么，解决什么问题

模块	典型内容	主要解决的问题
模型交互与输出控制	Structured Output、Function Calling、模型路由、上下文入口	把开放式生成转成系统可消费的决策与数据
编排与行动 Loop	状态机、路由、计划、重试、停止、子任务、多 Agent	让任务按观察结果持续推进，同时保持边界和预算
工具、Skills 与 Workspace	MCP/API、Shell、文件、浏览器、Skill、Sandbox、快照	让模型获得行动力和过程知识，并把副作用限制在可控环境
会话、状态与记忆	Session、Progress、Checkpoint、短期/长期记忆、Handoff	在多轮、多会话和长任务中维持连续性
服务化与消息边界	API/Worker、外部消息收发、Streaming、Cancel、Concurrency	把 Agent 作为长期运行的信息系统能力对外提供
可观测、验证与 Eval	Event logs、Trace、解析与可视化、Tests、Policy、Eval、Regression	还原过程、独立验收、归因失败、持续改进

多 Agent 协作不是一个孤立的新世界，它主要扩展“编排与行动 Loop”“会话与状态”“消息边界”和“验证”四个模块。长期记忆与自进化则主要扩展“状态与记忆”“可观测与 Eval”。

Harness 覆盖哪些层次

公开讨论经常把某个组件直接叫作 Harness。分层之后更容易判断它的角色。

覆盖层次：从模型到组织，不要把组件和整体混在一起

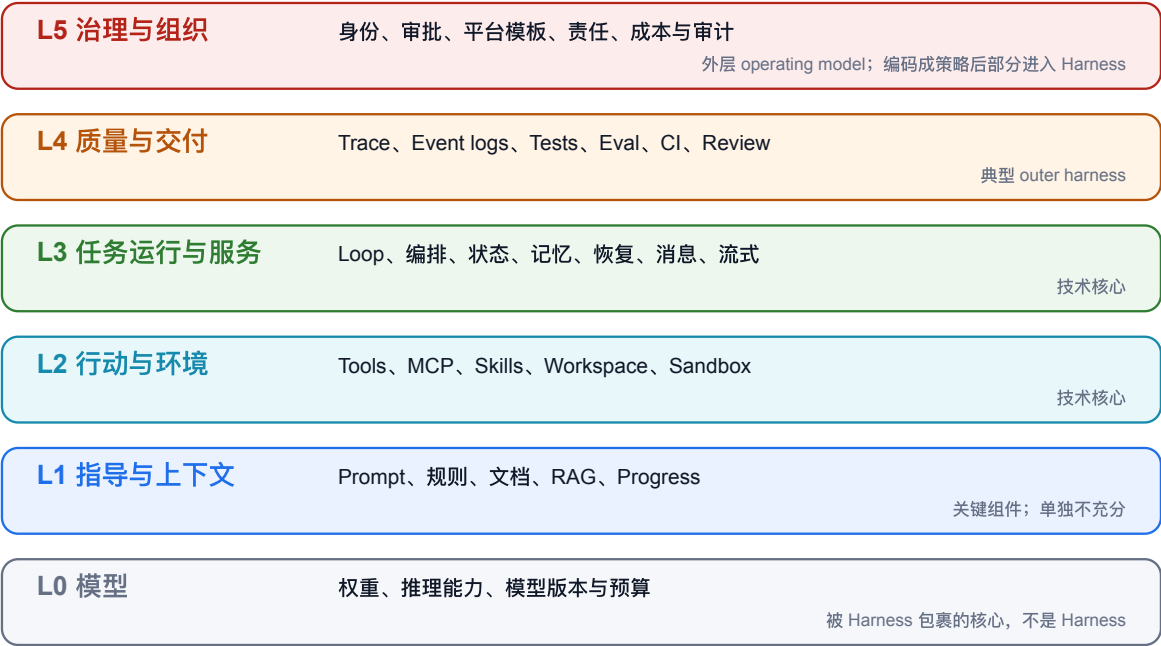


图 8：从模型到组织的六层结构。

分层解释

- **L0 模型**：能力基础，但不是 Harness 本体；
- **L1 指导与上下文**：Prompt、文档、Skills、RAG、Progress，是重要组件，单独不充分；
- **L2 行动与环境**：Tools、MCP、Workspace、Sandbox，让模型能感知和改变现实状态；
- **L3 任务运行与服务**：Loop、编排、状态、记忆、恢复、消息和 Streaming，负责持续执行；
- **L4 质量与交付**：Event logs、Trace、Tests、Eval、CI 和 Review，让结果可见、可验、可回归；
- **L5 治理与组织**：身份、审批、成本、审计、职责和平台模板。它通常是更外层的 operating model，只有被编码成运行策略的部分才进入 Harness。

完整 Harness 不是“每一层都塞满功能”。更准确的判断是：它至少形成了行动、状态、控制与验证的闭环，并且厚度与任务风险相匹配。

它不是 Prompt Engineering，也不只是 Agent Loop

对象	主要问题	能否独立保障 Agent 任务
Prompt Engineering	怎样表达指令，让模型更可能理解和遵循	不能；它影响行为，但不强制执行
Context Engineering	当前一步让模型看到哪些信息	不能；是核心子系统，但不处理全部执行和验证
Agent Loop / ReAct	模型怎样在观察与行动之间迭代	不能；没有状态、权限和验证时只是循环算法
MCP / ACP / Tools	能力和消息怎样被标准化暴露	不能；是接口或协议
Agent Framework / SDK	提供构建 Agent 的原语	不能自动保证；需要具体配置和运行机制
Agent Harness	模型怎样在真实环境中受控、持续、可恢复地完成任务	是讨论的核心对象
Agent-first Operating Model	人与 Agent 怎样分工，组织怎样交付	更外层的社会技术系统

可以把三者的区别压缩成一句话：

- Prompt 在说“希望模型怎样做”；
- Loop 在说“模型什么时候继续想和行动”；
- Harness 在说“这件事在哪里运行、能做什么、状态怎样保存、结果怎样验收、出错怎样恢复”。

每个场景都要考虑 Harness，但不需要同样厚

一个简单的只读问答，本身已经有一层很薄的 Harness：消息历史、超时、格式和日志。一个需要跨系统写入、运行数小时、能够取消和恢复的 Agent，则需要明显更厚的运行与控制层。

决定 Harness 厚度的主要因素不是“用了多大的模型”，而是：

- 任务持续多久、跨多少会话；
- 行动是否有不可逆副作用；
- 路径是否需要动态探索；
- 结果是否容易被独立验证；
- 失败后的成本和责任有多高。

Harness 不是越厚越好：厚度跟任务风险和任务跨度有关



图 9：低风险短任务可以薄，高风险长任务需要完整的运行、验证和治理能力。

三个工程观察：为什么不能只看模型

下面三个观察分别来自软件修复实验、长任务实践和 agent-first 仓库实践。它们放在一起看，能把一件事说清楚：模型能力很重要，但 Agent 的任务表现不是模型单独决定的。

观察 1：固定模型下改变接口，任务结果会跟着变

SWE-agent: Agent-Computer Interface 实验

来源说明 John Yang 等，NeurIPS 2024, *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*

- **研究问题：**在不修改模型权重的情况下，给 Agent 设计不同的文件查看、搜索、编辑、反馈和上下文接口，会不会改变软件修复表现？
- **相对可比的设置：**论文在 SWE-bench Lite 的 300 个真实 GitHub Issue 上使用同一个 GPT-4 Turbo，比较自定义 ACI 与只使用普通 Shell 的 Agent。
- **主要结果：**自定义 ACI 解决率为 **18.0%**，Shell-only 为 **11.0%**，论文报告相对提升约 **64%**。
- **关键消融：**移除专用编辑器后降到 **10.3%**；保留编辑器但去掉 Lint 阻断后为 **15.0%**；文件视图从 100 行改成完整文件后从 **18.0%** 降到 **12.7%**；把“最近 5 个观察”改成完整历史后从 **18.0%** 降到 **15.0%**。

这个观察说明：在模型与任务集相同的情况下，行动接口、反馈格式、上下文窗口和确定性检查会实质改变 Agent 的可实现能力。

阅读入口：[SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering](#)

观察 2：强模型 + Loop + Compaction 仍无法自然维持长任务

Anthropic：跨上下文长任务实验

来源说明 Anthropic Engineering, 2025, *Effective harnesses for long-running agents*

- Opus 4.5 已运行在 Claude Agent SDK 中，具备工具和上下文压缩；系统跨多个上下文窗口循环执行高层 Web 应用任务。
- 失败并不是“模型完全不会写代码”，而是一次做太多、交接失真、重复劳动、环境带病继续、提前宣布完成。
- Anthropic 通过 initializer、功能清单、Progress、Git、[init.sh](#)、浏览器端到端测试和“每次只做一个功能”来外部化任务连续性。

这个观察说明：长任务的持续性需要外部工件和运行规则；上下文压缩不能等同于项目状态管理。

阅读入口：[Anthropic — Effective harnesses for long-running agents](#)

观察 3: OpenAI 的 Agent-first 仓库把“模型看不见的东西”改造成系统对象

OpenAI: agent-first 仓库实践

来源说明 Ryan Lopopolo / OpenAI Engineering, 2026, *Harness engineering: leveraging Codex in an agent-first world*

- **背景**: 团队从空仓库开始, 让 Codex 生成产品代码、测试、CI、文档、可观测和内部工具; 人主要负责意图、环境和反馈回路。
- **一次失败的设计**: 团队最初把大量规则放进一个巨型 **AGENTS.md**。问题很快出现: 它挤占上下文、让所有规则失去重点、迅速陈旧, 并且难以机械检查。
- **后来的结构**: 约百行的 **AGENTS.md** 只做目录; 详细事实进入版本化 **docs/**; 复杂任务使用带进度和决策记录的 execution plan; Lint 和 CI 检查文档结构、新鲜度和交叉链接; 日志、指标和 Trace 也被暴露给 Agent 查询和验证。

这个观察说明: 仓库知识、计划、观察信号和质量约束不是“给模型多写几句提示”, 而是需要被设计成 Agent 可发现、可执行、可验证的系统资产。

阅读入口: [OpenAI — Harness engineering: leveraging Codex in an agent-first world](#)

三个观察放在一起, 能得到什么

观察	主要控制变量	改变的系统因素	说明什么
SWE-agent 接口实验	同一模型、同一 benchmark	编辑/搜索接口、视图大小、Lint、历史管理	接口和控制会直接影响任务表现
Anthropic 长任务案例	强模型 + SDK + Loop + Compaction 已存在	跨会话工件、增量策略、启动与端到端验证	长任务需要外部状态和可验证交接
OpenAI agent-first 仓库	同一真实仓库长期迭代	文档结构、计划、CI、可观测、Agent 可读性	仓库环境本身会成为 Harness 的一部分

共同结论不是“只看 Harness、不看模型”，而是：**Agent** 的任务表现是模型、接口、上下文/状态、控制机制与外部环境共同产生的系统属性。

从三个观察得到的四条工程结论

以下四条不是口号，而是从前面的接口实验、长任务案例和仓库实践里抽出来的工程判断：

1. 任务从可验证的契约开始，而不是只从一段 Prompt 开始。

长任务案例用端到端功能清单定义“什么算完成”，agent-first 仓库把复杂工作写成包含进度和决策记录的 execution plan。由此可推：目标、范围、证据、预算、权限和升级条件应当成为外部任务对象。

2. 上下文窗口不是任务状态。

长任务案例里，Compaction 仍会让新会话猜测上一会话；Progress、Git 和可启动基线才承担连续性。agent-first 仓库同样把计划和事实放在版本化仓库内。由此可推：计划、进度、决策、Workspace 和证据需要独立持久化，不能只保存在聊天历史里。

3. 能用确定性机制表达的约束，不应只写成自然语言。

接口实验里的 Lint 阻断比“请不要写出语法错误”更直接；四问法中的 T4 也强调控制不能依赖模型自觉；agent-first 仓库则用 Lint/CI 校验文档和架构。由此可推：Schema、Policy、Tests、Linters、权限和状态断言应尽量在模型之外执行。

4. 重要失败要进入可重复的回归闭环。

重复错误适合固化为规则、工具或程序化检查；团队外层也需要同时有 Guides 与 Sensors：先预防，再观察和自纠。由此可推：保存 Trace 和环境结果，先定位失败属于模型、工具、上下文、状态还是验证，再修改最接近根因的 Harness 层，并把该失败加入 Eval/回归集。

这四条也是理解任务契约、长期记忆、运行时控制、Eval、自进化和多 Agent 协作的共同前提。

所以 Harness 工程应该掌握什么

这张 Harness 地图不是把所有功能堆成清单。它更像一张能力检查表：凡是 Agent 要从“能回答”走到“能稳定完成任务”，都会落到其中一块系统责任上。

顺着这张图看，Harness 工程至少要掌握六组能力：

能力面	覆盖内容
请求、输出与决策	单次模型请求、结构化输出、路由、任务分解、反思、多步编排
工具、MCP 与 Skills	工具调用回路、MCP 生态接入、Skills 撰写、选择、执行与校验
运行环境与安全边界	Shell、文件、浏览器、Sandbox、权限、审批、幂等和风险分级
状态、记忆与协作	Workspace、Checkpoint、长期记忆、自我成长、多 Agent 分发与回收
服务化、消息与用户入口	API/Worker、流式输出、外部 IM、消息入口、取消、并发与用户可见过程
观测、Eval 与治理	Event logs、Trace、过程回放、Eval、成本、安全、审计和上线形态

自我成长和多 Agent 协作是其中两条重要分支，但不能代表全部 Harness 工程。更完整的判断是：模型请求怎样变成可执行任务，任务怎样接入工具和环境，过程怎样保存状态、交接、服务化、被观察和被治理。

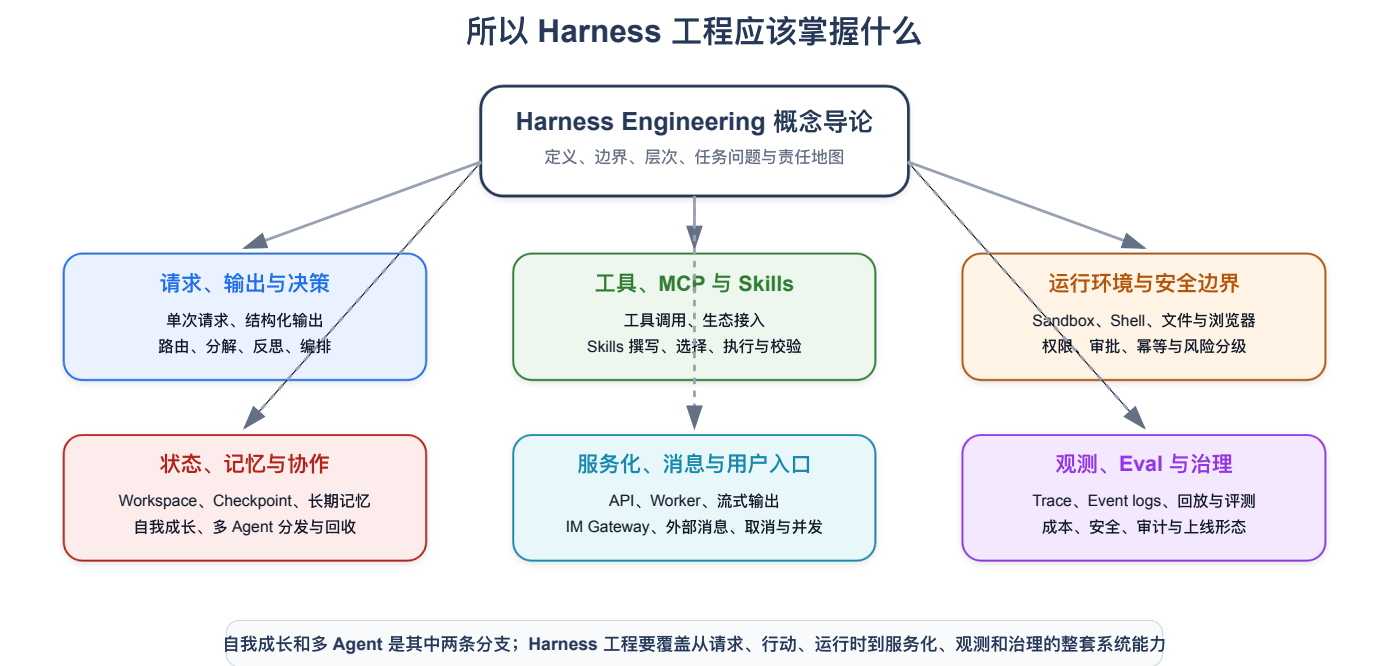


图 10：Harness 工程要掌握多个能力面，自我成长和多 Agent 只是其中两条分支。

总结

1. **Agent** 的表现不是模型单独的属性，而是模型、Harness 与环境共同作用的结果；
2. **Harness Engineering** 没有唯一稳定定义，分歧主要来自分析单位不同：运行时、项目外层、工程方法或组织系统；
3. 技术核心是把模型输出变成受控任务执行：行动、工作空间、状态、权限、服务、验证、观测和恢复需要形成闭环；
4. **组件不等于整体**：Prompt、MCP、Skill、Sandbox、Agent Loop、文档规范和 Eval Harness 都很重要，但单独看通常不构成完整 Harness；
5. **真正的落地问题发生在任务生命周期里**：输出会产生副作用、状态会跨会话、最终文本不等于结果、失败需要复现和回归；
6. **Harness 的厚度跟任务风险匹配**：从单次请求到 Function Calling、Agent Loop、长任务和多 Agent，运行与控制责任会逐步增加。

模型提供基础智能；Harness 把这种智能接入系统，变成可以持续行动、被约束、被观察、被验证和被改进的任务能力。